

Pearagon Task 3: Hubspot Architecture for Wane Enterprises

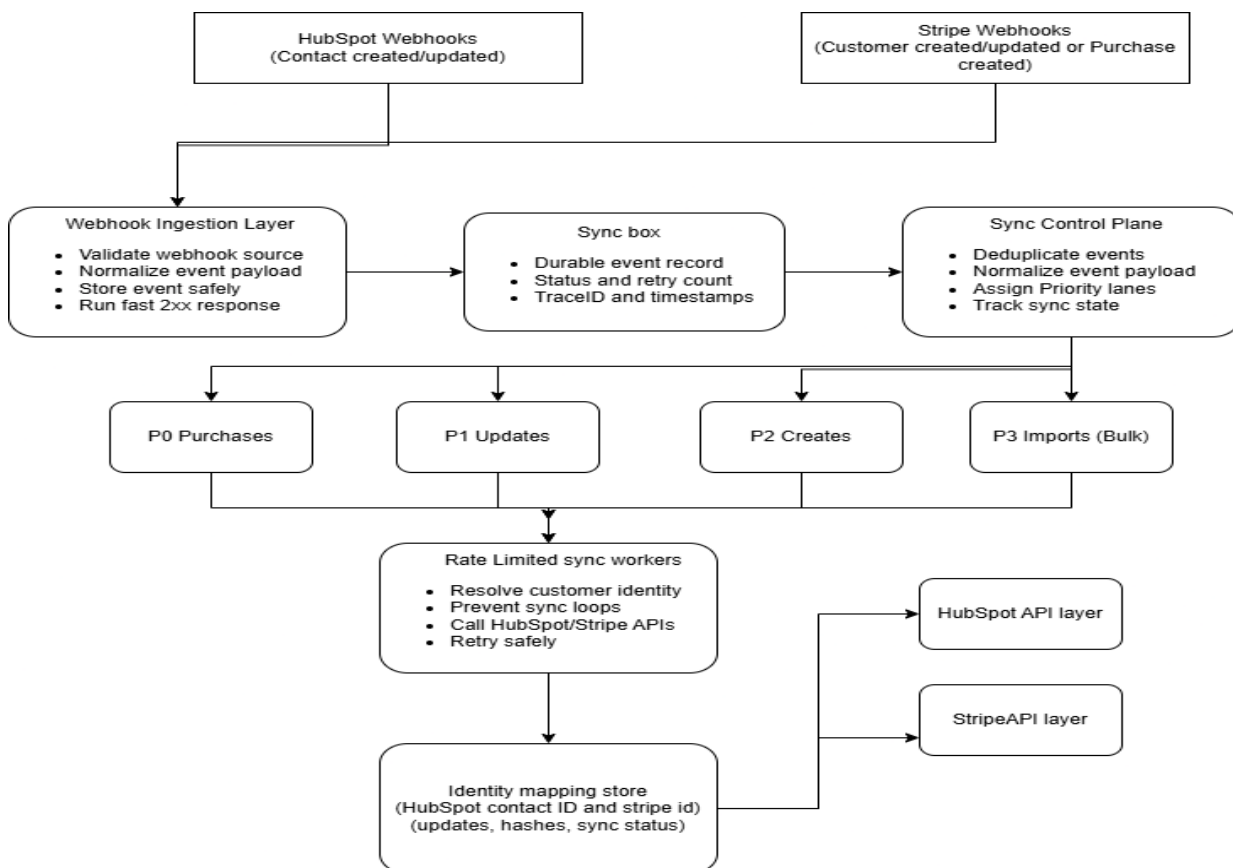
1. Overview

As far as I have analyzed this task, it wasn't about implementing the simplest possible interaction between the two APIs: one calls another back and forth. It is about managing two independent systems' consistency when both can modify customer information, webhooks come sporadically in large numbers, and both APIs are limited in their capacity.

My proposed architecture for solving the problem is an event-driven synchronization platform. The primary point of it would be that webhook processors need to capture and persist events promptly, but actual synchronization will be managed by dedicated workers. This approach includes two-way customer and contact management (create/update), Stripe purchases capturing as HubSpot custom events, retries, rate limit mitigation, loops avoidance, consistency, and a feasible solution for 30-second visibility guaranteed to Wane Enterprises.

2. Proposed Architecture

The main architecture would include these pieces:



3. Why I Would Design It This Way

The first thing I considered was whether direct webhook-to-API calls would be enough. For small traffic, that approach could work. For example, if a HubSpot contact is created, the webhook handler could immediately call Stripe and create a customer.

However, this becomes risky once we consider the real requirements:

- HubSpot can create or update 10,000 contacts in one minute during imports
- Stripe has a limit of 100 requests per second
- HubSpot has a limit of 15 requests per second
- Webhooks are sent one event at a time
- Both systems can trigger updates back into each other
- The customer expects changes to appear quickly in the other system

Because of this, I would not make webhook handlers responsible for API synchronization. I would keep webhook ingestion fast and simple, then let workers process events at the correct rate.

This gives us better control over retries, duplicates, rate limits, failures, and monitoring.

4. Webhook Ingestion Layer

The webhook ingestion layer receives events from both HubSpot and Stripe.

Its responsibilities would be:

- Validate the webhook signature or source
- Normalize the incoming payload into a common internal format
- Create one Sync Inbox record for each webhook event
- Return a fast success response
- Avoid calling HubSpot or Stripe APIs directly

This is important because the assumptions say each webhook contains one event. During a large HubSpot import, the system may receive thousands of individual webhook calls. The ingestion layer should accept those events quickly and safely without becoming blocked by downstream API limits.

5. Sync Inbox / Event Store

Every incoming webhook event should be stored before it is processed.

Example fields:

event_id, source_system, event_type, source_entity_id, payload, status, priority_lane, retry_count, received_at, processed_at, last_error, trace_id

Why Is the Sync Inbox Important? It is important since it creates a log of events for integration that is separate from the queue. This way, while the queue provides information on what should be processed currently, the Sync Inbox gives information on what took place, when, and why that particular event failed. The importance of such an event will become more evident when we need to replay any missed event or respond to any consumer complaints.

6. Identity Mapping Store

The integration needs a dedicated mapping store between HubSpot contacts and Stripe customers.

Example fields:

hubspot_contact_id, stripe_customer_id, normalized_email, normalized_phone, last_synced_hash, last_write_source, last_write_trace_id, sync_state, created_at, updated_at

I would not rely only on email for matching because emails can change. Phone numbers can also change or be formatted differently. The mapping store becomes the integration's source of truth for knowing which HubSpot contact and Stripe customer represent the same person.

This avoids duplicate customer creation and makes updates safer.

7. Sync State Tracking

Each mapped customer/contact pair should have a clear sync state.

Example states:

UNMAPPED, MAPPED, SYNC_PENDING, SYNCED, RETRYING, FAILED_NEEDS_REVIEW

This makes the system easier to operate. Instead of only knowing that a queue message failed, the team can understand the actual business state of the record.

For example:

- This customer is synced.
- This customer is mapped but retrying.
- This purchase is waiting for a HubSpot contact mapping.
- This record failed validation and needs review.

8. Priority Lanes

I would not process every event in one generic queue. I would separate events by business priority.

P0: Stripe purchases to HubSpot custom events

P1: Contact/customer updates

P2: Normal creates

P3: Bulk/import creates

Purchases happen at P0 as they reflect ongoing activities. If there is a delay in a purchasing event, then what this means is that the sales rep is viewing a contact in the HubSpot but there was no purchase event recorded in HubSpot for something that has taken place; this is what can quickly create mistrust in the integration.

The updates happen at P1, and this is based on the reality that incorrect contact data will only become compounded if the update does not take place.

Normals will happen at P2 as the problem of not having a newly created contact recorded in Stripe for an additional second is much less damaging than failing to record a purchase.

P3 can be reserved for Bulk imports.

9. Event Compaction

During large imports, the same contact may generate multiple events close together.

Example:

- Contact created
- Email updated
- Phone updated
- Name updated

If the integration processes each intermediate event separately, it wastes API calls and rate-limit capacity.

Instead, the control plane should compact events where possible and sync the latest known state.

This reduces:

- Unnecessary API calls
- Worker load
- Queue pressure
- Stale writes
- Rate-limit usage

This is especially important in the HubSpot-to-Stripe import path because Stripe writes are limited to 100 requests per second and Stripe is assumed to support only single-customer operations.

10. Data Synchronization Flows

1. HubSpot Contact Created to Stripe Customer Created

Flow:

1. HubSpot sends a contact-created webhook.
2. The webhook is validated and stored in the Sync Inbox.
3. The event is routed to the create or import lane.
4. A worker checks the Identity Mapping Store.
5. If a mapping already exists, the event is skipped.
6. If no mapping exists, the worker creates a Stripe customer.
7. The mapping store is updated with the HubSpot Contact ID and Stripe Customer ID.
8. The event is marked complete.

Important note: this path writes into Stripe. Since the assumptions only provide a single-customer Stripe API and a 100 requests per second limit, this path must be processed at a controlled rate.

2. Stripe Customer Created to HubSpot Contact Created

Flow:

1. Stripe sends a customer-created webhook.
2. The event is stored in the Sync Inbox.
3. A worker checks the mapping store using Stripe Customer ID, email, and phone.
4. If no mapping exists, the worker creates a HubSpot contact.
5. The mapping store is updated.

HubSpot supports batch creation of up to 100 contacts. I would use that only for prevalidated bulk work, not blindly for every real-time event.

The reason is that the problem statement says one invalid contact can fail the entire batch with poor error reporting. For real-time flow, single-contact writes are safer. For larger Stripe-to-HubSpot backfills or controlled bulk jobs, prevalidated batches can improve throughput.

3. HubSpot Contact Updated to Stripe Customer Updated

Only these fields need to sync:

Email
First Name
Last Name
Phone Number

Flow:

1. HubSpot sends an update webhook.
2. The event is stored.
3. The worker finds the mapped Stripe customer.
4. The worker compares the new field values with the last synced hash.
5. If nothing meaningful changed, the event is skipped.
6. If values changed, the worker updates the Stripe customer.
7. Sync metadata is updated.

4. Stripe Customer Updated to HubSpot Contact Updated

Flow:

1. Stripe sends a customer update webhook.
2. The event is stored.
3. The worker finds the mapped HubSpot contact.
4. The worker compares the synchronized fields.
5. If values changed, the worker updates the HubSpot contact.
6. Sync metadata is updated.

Stripe customer updates are only about 10 per day, so this path should comfortably stay within the 30-second target.

5. Stripe Purchase to HubSpot Custom Event

Flow:

1. Stripe sends a purchase webhook.
2. The event is routed to the P0 purchase lane.
3. The worker finds the mapped HubSpot contact.
4. The worker creates a custom event on that HubSpot contact.
5. The event is marked complete.

If a purchase arrives before the mapping exists, the system should retry briefly. If the mapping is still missing, the event should move to the retry queue and eventually the dead-letter queue if unresolved.

11. Rate Limiting and Scalability

Workers can scale horizontally, but they must still respect the vendor API limits.

The system should enforce:

Stripe: 100 requests per second

HubSpot: 15 requests per second

Although we can scale horizontally for queue depth, horizontal scaling alone will not address the issue of rate limiting, if ten workers ignore rate limiting, they will all receive 429 errors. A Redis implementation of the token bucket approach on a fleet-wide level guarantees that Stripe and HubSpot remain within their own limits globally, regardless of how many workers there are.

However, the critical bottleneck in terms of direction must be noted here. In the case where Stripe writes to HubSpot are backed up, we can batch together 100 contacts in each API call, thereby multiplying the throughput of the latter by 100. This is straightforward to control. There is no equivalent direction in the case of HubSpot writes to Stripe; Stripe can only work with a single customer at a time, which means that each contact added via a bulk process will

result in an API call to Stripe. At 100 requests per second, that becomes our hard limit, and we cannot do anything other than prioritize live traffic.

12. 30-Second SLA Handling

For normal traffic, this architecture should meet the 30-second requirement.

Normal traffic includes:

- HubSpot average contact changes: about 1,000 per day
- Stripe customer creates: about 1,000 per day
- Stripe customer create peak: about 100 per minute
- Stripe customer updates: about 10 per day
- Stripe purchases: about 10,000 per day, peaking around 150 per minute

This amount is easily accommodated by the architectural design; the queues smooth out the variability, priority channels safeguard the real-time data traffic, and the pool of workers ensures that the queue is drained without reaching the ceiling of the rates. The hard case is the HubSpot import burst: 10,000 HubSpot contacts in one minute.

In that direction, HubSpot-created contacts must be created or updated as Stripe customers. Stripe has a 100 requests per second limit and is assumed to support only single-customer APIs.

The minimum possible time is: $10,000 / 100 = 100$ seconds

So, I would be clear that the integration can ingest all events immediately and keep live traffic protected, but it cannot make all 10,000 records fully visible in Stripe within 30 seconds under the given Stripe limit.

To get closer to the SLA, I would:

- Put imports in a lower-priority lane
- Keep purchases and updates ahead of imports
- Compact multiple events for the same contact
- Process imports at the maximum safe Stripe rate
- Monitor oldest event age and SLA risk
- Use scheduled pre-sync for planned imports when possible
- Request a higher Stripe rate limit for known import windows

To truly guarantee under 30 seconds for a 10,000-contact HubSpot import, the system would need about:

$10,000 / 30 = 334$ Stripe writes per second

Realistically, I would ask for 400 to 500 Stripe writes per second to leave room for retries and normal live traffic.

13. HubSpot Batch API Handling

The assumptions include a HubSpot batch API that can create up to 100 contacts at a time.

I would use this carefully.

Good use case: Stripe-to-HubSpot bulk creation after validation

Risk: One invalid contact can fail the whole batch

Handling:

1. Validate email and phone before batching.
2. Try a batch of up to 100 records.
3. If the batch fails, split it into smaller batches.
4. Continue splitting until the bad record is isolated.
5. Send only the invalid record to the dead-letter queue.
6. Continue processing valid records.

This gives us the benefit of batching without allowing one bad record to block many good records.

14. Error Handling and Retry Strategy

I would classify errors before retrying.

Retryable errors:

429 rate limits, 5xx vendor errors, network timeouts, temporary service failures

Non-retryable errors:

invalid email, invalid phone number, missing required fields, malformed payloads, permanent validation failures

Retry schedule example:

- Attempt 1: 5 seconds
- Attempt 2: 30 seconds
- Attempt 3: 2 minutes
- Attempt 4: 10 minutes

Then move to dead-letter queue

Retries should use jitter so all failed events do not retry at the same moment.

If HubSpot or Stripe repeatedly returns 429 or 5xx responses, the system should slow down or pause that lane temporarily using a circuit breaker. This prevents the retry system from making an outage worse.

15. Dead-Letter Queue

After retry exhaustion, events should move to a dead-letter queue.

DLQ records should include:

event_id, source_system, target_system, payload, error_message, attempt_count, last_attempt_at, replay_status

The operations team should be able to inspect failed events, fix data if needed, and replay them safely.

16. Loop Prevention

There is an inherent feedback loop issue in two-way syncs. A change in HubSpot causes a write operation on Stripe, causing a Stripe webhook to fire, causing another HubSpot write, and so on.

The fix for this is straightforward. Any sync write should leave sufficient metadata in the target record such that, when the echo webhook comes back around, the worker recognizes it wrote the field itself, and simply ignores it. Hashing the field content also helps with the same problem, as there is no point touching the other system if none of those fields were actually modified since the last sync.

17. Consistency Model

This integration should use eventual consistency.

Strict transactional consistency is not realistic because HubSpot and Stripe are separate external systems. The system cannot commit to both systems in one transaction.

On the contrary, this is achieved through durable events persistence, idempotent processing, field hashing, retries, and reconciliation, where the identity map is the common thread between all these processes and dead letter queues ensure that no message fails silently. The objective of this is to make sure that all successfully processed events get into their proper state or else fail visibly.

18. Reconciliation Worker

Drift is bound to happen even in an effective webhook-based system. Webhooks will fail when the vendor experiences an outage, manual updates will circumvent the integration process altogether, and edge cases with regard to mapping rules will cause contacts to get stuck now and again.

The role of this job is to run periodic checks for HubSpot contacts with missing Stripe mapping, Stripe customers with missing HubSpot mapping, pairs where field hashes do not match anymore, Sync Inbox items that have been stuck in a non-terminal state for too long, dead letter items that need replay, and purchases whose mapping still hasn't happened.

19. Security Requirements

Each webhook endpoint must first verify the incoming signature before even touching the payload, since all other requests without signatures should be rejected out of hand. All services-to-service traffic and communication with HubSpot and Stripe APIs must occur over HTTPS/TLS.

Keys and credentials should never be hardcoded or set as environment variables. They should go into a separate secrets manager accessible only to services which actually need them.

Since the store and audit store contain personal contact information, both should be encrypted when at rest. Logging needs special attention in that regard: most often, no full raw payloads should ever hit the logs, and fields such as email and phone number should be scrubbed of personally identifiable information unless logging for debugging purposes. Service accounts and worker accounts should operate on the principle of least privilege.

20. Monitoring and Alerting

The most critical measure to track is the age of the oldest event by lane, as this gives immediate insight into the state of operations. While queue depth is an important metric for context, it does not provide insight into the aging time of each event.

P50, P95, and P99 latencies for sync will provide visibility on typical performance compared to outlier performance, while 30-second SLA breaches will reflect your promise to the customers. On the vendor side, tracking the error rate for both HubSpot and Stripe will give a distinction between rate limit problems and an outage.

Dead-letter queue growth is one of the metrics that matter a lot on an operational level, where a slow increase in DLQ count goes unnoticed until the drift gets out of hand.

Alerts must be set up whenever there is a purchase event P0 older than ten seconds, P1 or P2 events older than thirty seconds, any increase in DLQ count, spike in 429 or 5xx errors, or excessive growth of the bulk import queue against worker speed.

21. Infrastructure Recommendation

Ingestion via webhook should utilize serverless functions due to its inherently spiky nature, where regular usage is low, but a bulk HubSpot import may suddenly cause an influx of up to 10,000 requests per minute. This can be easily handled by serverless.

Workers are an entirely separate case where serverless will work only if the underlying platform provides fine-grained concurrent execution capabilities allowing rate limiting to be enforced on all instances at once. Otherwise, a set of small VPSs may provide more certainty when you have to work within a strict 15 requests per second limitation imposed by HubSpot.

Identity resolution in the mapping store requires a relational database such as Postgres instead of just a simple key-value store due to multi-key lookups and transaction support for synchronization states required. Failure to update a single mapping record leads to duplicates in the customers list. Redis is used for the state of the rate limiter since the token bucket mechanism has to be shared between workers in real-time, and using any other storage will incur excessive overhead for checks. API keys and webhook signatures are stored securely in the cloud secrets management system.

22. Programming Specifications

This separation is deliberate based on concerns. The ingestion module does not talk to vendor APIs at all; it is purely responsible for ingesting and storing. The SyncRouter is responsible for deduplication and prioritization after ingestion and before execution. The adapters provide all the vendor-specific implementation details such that changes to the HubSpot API, for example, will not affect the worker and router implementations in any way. The RateLimiter module, on the other hand, is common infrastructure code called by all workers before making any outgoing requests.

The implementation should include these modules or services:

WebhookIngestionService

- validate webhook
- normalize event
- create Sync Inbox record
- enqueue event pointer

SyncRouter

- deduplicate event
- compact stale events
- assign priority lane
- update sync state

SyncWorker

- resolve identity
- prevent loops
- call target API
- update mapping
- mark event complete

HubSpotAdapter

- search contact
- create contact
- batch create prevalidated contacts
- update contact
- create custom event

StripeAdapter

- search customer
- create customer
- update customer

RateLimiter

- enforce HubSpot 15 requests per second
- enforce Stripe 100 requests per second
- reserve capacity by priority lane

ReconciliationWorker

- detect drift
- repair mappings
- replay safe failures

23. Conclusion

I would recommend an identity-first, event-driven synchronization platform between HubSpot and Stripe.

The important design decision is separating event ingestion from synchronization work. This allows the system to receive webhooks quickly, process work safely, respect rate limits, avoid duplicate records, prevent sync loops, and recover from failures.

This architecture supports all required data flows and gives Wane Enterprises strong monitoring, retry handling, security, and scalability. It should meet the 30-second target for normal real-time activity. For the specific HubSpot import burst of 10,000 contacts in one minute, the system can ingest and prioritize safely, but full visibility in Stripe cannot be guaranteed under 30 seconds unless Stripe throughput is increased, scheduled imports are pre-synced, or bulk import workloads receive a separate SLA.